

QueryLens: A Validation-Aware SQL Query Analysis Tool for Performance Optimization

COMP8047 Major Project
Parshotam Parmar
A00590956
British Columbia Institute of Technology
Date: May 24, 2026
Supervisor: Tejinder Randhawa

Table of Contents

1 Introduction	1
1.1 Student Background.....	1
1.2 Project Description	2
1.3 Essential Problems	2
1.4 Goals and Objectives	3
2.1 Background.....	4
2.2 Project Statement.....	4
2.3 Possible Alternative Approaches	5
2.4 Chosen Solution	5
2.5 Details of Design and Development.....	6
2.5.1 Feasibility Assessment	6
2.5.2 Design Diagrams	6
2.5.3 System Architecture Diagram.....	7
2.5.4 Sequence Diagram.....	8
2.5.5 Data Flow Diagram.....	9
2.5.6 Static Analysis Engine	9
2.5.7 Execution Plan Analysis	10
2.5.8 Correlation and Validation Engine	11
2.5.9 Reporting and Artifact Generation	11
2.6 Installation Manual.....	12
2.6.1 Workstation Requirements	12
2.6.2 Python Environment Setup.....	12
2.6.3 SQL Server Configuration	12
2.6.4 Dataset Setup.....	13
2.6.5 Configure db_config.py	13
2.6.6 Running QueryLens	13
2.7 User Manual.....	13
2.7.1 Offline Mode Workflow	14
2.7.2 Live Mode Workflow	14
2.7.3 Generated Outputs	14
2.7.4 Common Errors	15

2.8 Testing Details and Results	15
2.8.1 Static Analysis Testing.....	15
2.8.2 Runtime Validation Testing.....	16
2.8.3 Reporting Validation	16
2.9 Implications of Implementation.....	16
2.10 Innovation.....	16
2.11 Complexity.....	17
2.12 Research in New Technologies	17
2.13 Future Enhancements.....	18
2.14 Timeline and Milestones.....	18
3. Implementation.....	19
3.1 Static Analysis Implementation.....	19
3.2 Runtime Analysis Implementation	19
3.2.1 Live Execution Plan Capture	20
3.3 Correlation Engine Implementation.....	21
4. Evaluation	21
4.1 Evaluation Environment	22
4.2 Overall Performance	22
4.3 False Positive Reduction	25
4.4 Precision Improvement	26
4.5 Ground Truth Validation	26
4.6 Rule-Level Performance.....	26
4.7 Validation Examples.....	27
5. Discussion	30
5.1 Strengths.....	30
5.2 Limitations.....	30
6. Conclusion	31
7. Professional and Academic Development	32
8. Lessons Learned.....	32
Appendix — Proposal Changes.....	32
References	33

1 Introduction

Modern database-driven applications depend heavily on efficient SQL query execution to ensure scalability and responsiveness. However, identifying performance issues in SQL queries remains a challenging task, particularly for developers who lack visibility into how queries behave at runtime. Traditional approaches to query analysis often rely on either static code inspection or runtime execution plan analysis but rarely integrate both perspectives in a unified framework.

Static analysis tools can identify common anti-patterns such as non-SARGable predicates, excessive data retrieval, and inefficient joins. However, these tools frequently generate false positives because they lack awareness of actual execution behavior. In contrast, execution plan analysis provides detailed insights into runtime performance but does not inherently explain the underlying causes of inefficiencies in the original SQL code. As a result, each approach offers only a partial view of query performance when used in isolation.

This project introduces QueryLens, a validation-aware SQL query analysis tool designed specifically for Microsoft SQL Server. QueryLens integrates grammar-based static analysis with execution plan evaluation to produce more accurate and actionable performance recommendations. In this context, validation-aware refers to the use of runtime execution evidence to confirm, suppress, or strengthen static SQL performance findings. By correlating static warnings with runtime evidence, the system reduces false positives and improves the reliability of optimization suggestions.

The motivation for this project stems from real-world experience working with SQL Server databases in a production environment, where inefficient queries and stored procedures frequently caused performance bottlenecks. Existing tools often produced noisy or incomplete recommendations, making it difficult to prioritize optimization efforts. QueryLens addresses this gap by combining static and runtime analysis into a single, coherent system.

The primary objective of this project is to demonstrate that integrating execution plan validation into static SQL analysis significantly improves precision and reduces false positive recommendations. Unlike conventional tools, QueryLens does not treat static and runtime analysis as independent processes but instead integrates them into a validation-aware framework that prioritizes confirmed inefficiencies over theoretical risks.

1.1 Student Background

My background in database systems and software development directly supports this project. I completed the Computer Systems Technology Diploma in the Information Systems option and am currently pursuing the Bachelor of Technology in Computer Systems, specializing in the Database option. These programs built my foundation in relational database design, query optimization, and back-end integration.

Professionally, I have worked for twelve years at a golf point-of-sale software company that maintains multiple SQL Server-based systems. In my current role as a Support Data Analyst, I investigate performance issues in production databases daily. I have learned how small inefficiencies—like poor indexing or unnecessary data retrieval—can cascade into user-visible

slowdowns. QueryLens grows directly from these experiences, turning practical insight into an automated tool that helps others find and fix similar problems quickly.

ANTLR was selected because its grammar-based parsing model aligned well with the project's requirement for structural SQL analysis and abstract syntax tree traversal. My experience working with complex stored procedures and query structures gives me a strong foundation for interpreting the abstract syntax trees that ANTLR produces. To minimize risk, I have allocated time in the early development phase to complete small prototype exercises using ANTLR's Python runtime—specifically lexical parsing, tree walking, and node extraction—before integrating it into QueryLens. With this preparation and the structured grammar provided, I will be prepared to use ANTLR effectively within the scope and timeline of this project.

1.2 Project Description

QueryLens is a Python-based SQL Server performance analysis tool that combines static SQL inspection with execution-plan validation. The system parses SQL queries, detects common performance anti-patterns, analyzes SQL Server execution-plan evidence, and generates validation-aware recommendations.

The tool is designed for developers, analysts, and database support staff who need practical SQL performance guidance without manually interpreting complex execution plans. Unlike static-only analysis, QueryLens uses runtime evidence such as scans, seeks, sorts, actual row counts, and actual execution counts to determine whether static findings are supported by observed execution behavior.

The final implementation uses ANTLR for grammar-aware T-SQL parsing, pyodbc for live SQL Server integration, XML execution-plan parsing for runtime analysis, and an HTML reporting layer for presenting findings, confidence levels, operator summaries, and rewrite suggestions. The project focuses specifically on Microsoft SQL Server to keep the scope controlled while allowing deeper runtime validation and index-aware analysis.

1.3 Essential Problems

The essential problem addressed by QueryLens is that SQL performance issues are often discovered too late. In production environments, inefficient queries may appear acceptable during development but cause delays when repeated across large datasets or concurrent users. These delays can affect reports, transactions, support workflows, and overall user confidence in the system.

Many teams also face a knowledge gap between writing SQL and interpreting how SQL Server executes it. Execution plans provide valuable diagnostic information, but they require specialized tuning knowledge. As a result, developers and analysts may recognize that a query is slow without clearly understanding whether the cause is unnecessary data retrieval, poor predicate design, missing indexing support, or an inefficient runtime operator.

QueryLens addresses this problem by making performance issues visible earlier and by translating execution-plan behavior into practical recommendations. Its purpose is to shift SQL tuning from reactive troubleshooting after users experience slowdowns to proactive validation during development and testing.

In many business environments, software performance is judged not by benchmarks but by how smoothly everyday work gets done. When transactions take longer than expected, reports lag, or systems freeze during busy periods, users often experience frustration without knowing the technical cause. These slowdowns can delay customer service, disrupt operations, and reduce confidence in the system—even when the underlying hardware and software appear to be functioning normally.

Behind many of these issues are small inefficiencies in how databases are queried. These problems are difficult to detect early because they often remain hidden during development and only appear under real workload conditions. Developers are typically expected to fix them after users are already affected. QueryLens is motivated by this gap between when performance problems begin and when they are finally understood.

By making database performance issues visible and explainable early in the development and testing process, QueryLens aims to shift performance tuning from a reactive task to a proactive practice that benefits both developers and end users.

1.4 Goals and Objectives

QueryLens will analyze SQL Server queries in two complementary ways:

1. **Static Analysis:** Reading SQL text to detect patterns known to reduce performance.
2. **Runtime Validation:** Reviewing the database engine's execution plan to confirm which issues truly impact speed. By connecting these layers, the system will avoid false alarms and will provide only confirmed, actionable recommendations. Its target users are software developers and data analysts who work with SQL Server but may lack deep tuning expertise. Each key goal maps directly to measurable outcomes:

QueryLens was designed around three primary goals:

Goal	Implementation	Validation Evidence
Detect inefficient SQL Patterns	Static analyzer using ANTLR-based feature extraction and rule evaluation	Static analysis outputs and static test logs
Validate runtime behaviour	SQL Server execution-plan parser and live plan capture through pyodbc	Runtime findings, actual row counts, actual executions, and operator summaries
Correlate findings and produce recommendations	Correlation engine with confidence scoring, suppression logic, and HTML reporting	Correlation matrix, validation report, false-positive log, and generated recommendations

Together, these goals support the project's main objective: to reduce unsupported static warnings by confirming or suppressing findings using execution-plan evidence.

2.1 Background

QueryLens was developed in the context of SQL Server-based business systems where application performance is closely tied to database query quality. In transactional environments, small inefficiencies in query design can produce noticeable slowdowns when repeated across high-volume workloads. This project is especially relevant to operational systems such as point-of-sale platforms, where delays in transaction processing, reporting, or database access can directly affect users and business workflows.

The project is also grounded in practical industry experience. Performance issues observed in production SQL Server environments frequently originated from patterns such as unnecessary column retrieval, non-SARGable predicates, poorly selective joins, and inefficient report queries. Although SQL Server exposes detailed execution plans, interpreting those plans requires a level of database tuning expertise that is not always available to software developers or support analysts. QueryLens was therefore designed to bridge this gap by turning execution plan evidence into understandable, development-oriented feedback. Existing SQL performance tools often separate static diagnostics from execution-plan analysis, requiring manual interpretation across multiple interfaces.

Within this operational context, the project addresses a real need: enabling earlier and more consistent identification of SQL performance risks before they escalate into production bottlenecks. QueryLens supports this goal by integrating static query inspection with runtime validation, allowing likely inefficiencies to be distinguished from confirmed performance issues.

2.2 Project Statement

QueryLens is scoped as a validation-aware SQL Server query analysis tool. The system analyzes standalone SQL workload files using grammar-aware static analysis, then validates detected findings against SQL Server execution-plan evidence. The implementation focuses on Microsoft SQL Server, Python, ANTLR-based T-SQL parsing, pyodbc database connectivity, XML execution-plan analysis, and automated artifact generation.

The core contribution of the project is the correlation layer between static SQL findings and runtime execution evidence. Rather than reporting every detected static anti-pattern as an equal warning, QueryLens evaluates whether runtime operators, actual row counts, actual execution counts, scans, seeks, sorts, joins, and index metadata support or weaken the static finding. Findings that are not supported by runtime evidence can be suppressed or marked as static-only, reducing misleading recommendations.

The project is not intended to be a full SQL optimizer, a replacement for SQL Server Management Studio, or a complete automatic tuning system. It operates as a controlled diagnostic prototype over a reproducible benchmark workload. Its purpose is to demonstrate that static SQL analysis becomes more useful when paired with runtime validation and evidence-based confidence scoring.

2.3 Possible Alternative Approaches

Several alternative approaches were considered before selecting the final design of QueryLens.

The first alternative was a **static-only SQL analysis tool**. This approach would examine SQL text for known anti-patterns such as `SELECT *`, functions in predicates, and complex joins, but would not consider execution plan behavior. A static-only solution would be simpler to implement and faster to run, but it would also produce a higher number of false positives because it would not be able to determine whether a detected issue affected execution behavior.

A second alternative was an **execution-plan-only analyzer**. This approach would focus entirely on SQL Server XML plans and operator-level behavior, such as scans, seeks, sorts, and joins. While this would provide strong runtime visibility, it would not explain the corresponding code-level source of inefficiencies. Such a system would be useful for database specialists, but less accessible to developers who need direct guidance about how to improve the original SQL text.

A third alternative was to rely on **existing tools such as SQL Server Management Studio or standalone linting utilities**. These tools provide partial support for performance analysis but typically emphasize either runtime diagnostics or code inspection rather than combining both in a single validation-aware workflow. They also tend to require significant manual interpretation and do not always prioritize issues according to supporting execution evidence.

For these reasons, the selected solution was a **hybrid validation-aware analysis system**. By combining static SQL analysis with execution plan correlation, QueryLens was designed to preserve the interpretability of code-level diagnostics while improving accuracy through runtime confirmation. This approach best aligned with the project's practical objective of reducing false positives and producing more reliable recommendations.

2.4 Chosen Solution

The chosen solution is a modular validation-aware analysis pipeline. QueryLens first parses SQL workload files using an ANTLR-based T-SQL grammar and extracts structural features such as selected columns, predicates, joins, subqueries, aggregations, ordering clauses, and table references. These features are evaluated by a static rule engine to identify potential SQL performance anti-patterns.

The second layer analyzes SQL Server execution plans. QueryLens supports both offline `.sqlplan` files and live execution-plan capture through `pyodbc`. In live mode, the system executes read-only workload queries against the validation database, captures actual execution plans using `SET STATISTICS XML ON`, and extracts runtime evidence such as scans, seeks, sorts, joins, actual row counts, and actual execution counts.

The third layer is the correlation engine. It compares static findings with runtime evidence, assigns confidence levels, suppresses unsupported findings, and distinguishes runtime-verifiable issues from static-only advisory findings. The final reporting layer generates JSON, CSV, graphical, and HTML artifacts so results can be inspected, reproduced, and included in evaluation.

2.5 Details of Design and Development

QueryLens is implemented as a multi-stage analysis pipeline, where each component produces structured artifacts consumed by subsequent stages. This design enables traceability between static detections, runtime validation, and final recommendations.

2.5.1 Feasibility Assessment

The feasibility of QueryLens was evaluated in technical, operational, and schedule terms.

From a **technical feasibility** perspective, the project was practical because all core technologies were available and well supported. Python provided a flexible environment for pipeline orchestration and artifact generation. ANTLR supplied a mature grammar-based parsing framework for T-SQL analysis, and SQL Server execution plans could be exported in XML format for deterministic parsing. The modular nature of the problem also made the system suitable for staged development, beginning with simple static rule detection and expanding toward runtime validation and reporting.

From an **operational feasibility** perspective, the required environment was modest and already available. Development and testing were performed on a single workstation using SQL Server Developer Edition, local datasets, and pre-generated execution plans. No external infrastructure, cloud services, or production data were required. This reduced both risk and implementation overhead.

From a **schedule feasibility** perspective, the project was manageable because it could be divided into incremental milestones. Early weeks focused on execution plan parsing and dataset engineering, followed by static analysis, correlation logic, evaluation, and reporting. This staged approach made it possible to validate progress continuously and reduce the risk of major late-stage integration failures. Over the course of development, the project expanded from prototype components into a consolidated end-to-end system totaling approximately 447 hours of effort.

Overall, QueryLens was feasible because it addressed a clearly bounded problem, used accessible technologies, and could be implemented iteratively with measurable intermediate deliverables.

2.5.2 Design Diagrams

The design of QueryLens is documented using three complementary diagrams. The system architecture diagram provides a high-level view of the major components and how data moves through the static analysis, runtime analysis, correlation, and reporting layers. The sequence diagram illustrates the order of execution during a full analysis run, especially the live SQL Server interaction used to capture execution plans. The data flow diagram focuses on how SQL files, execution plans, findings, and generated artifacts move through the system.

Together, these diagrams show both the structural design of QueryLens and the runtime workflow used to produce validation-aware results.

2.5.3 System Architecture Diagram

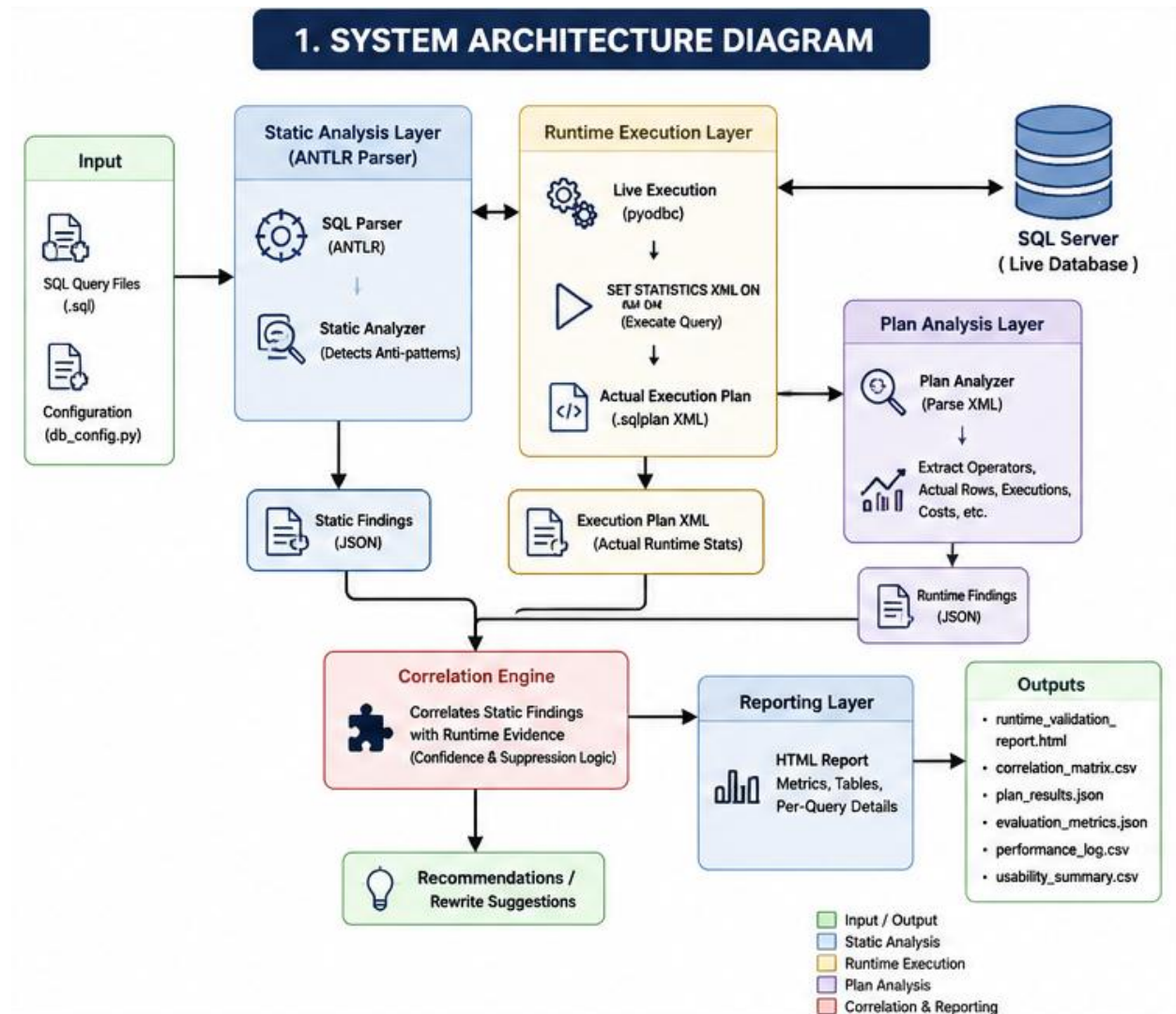


Figure 1: High-level QueryLens system architecture showing SQL parsing, static analysis, runtime execution-plan analysis, correlation validation, and artifact generation workflow.

The architecture separates QueryLens into four major layers. The static analysis layer parses SQL input and produces structured static findings. The runtime execution and plan analysis layers connect to SQL Server, capture or load execution plans, and extract runtime evidence such as operators, row counts, and execution counts. The correlation engine combines these two sources of evidence to determine whether findings are confirmed, suppressed, or treated as static-only. The reporting layer then converts the results into human-readable reports and reproducible artifacts.

2.5.4 Sequence Diagram

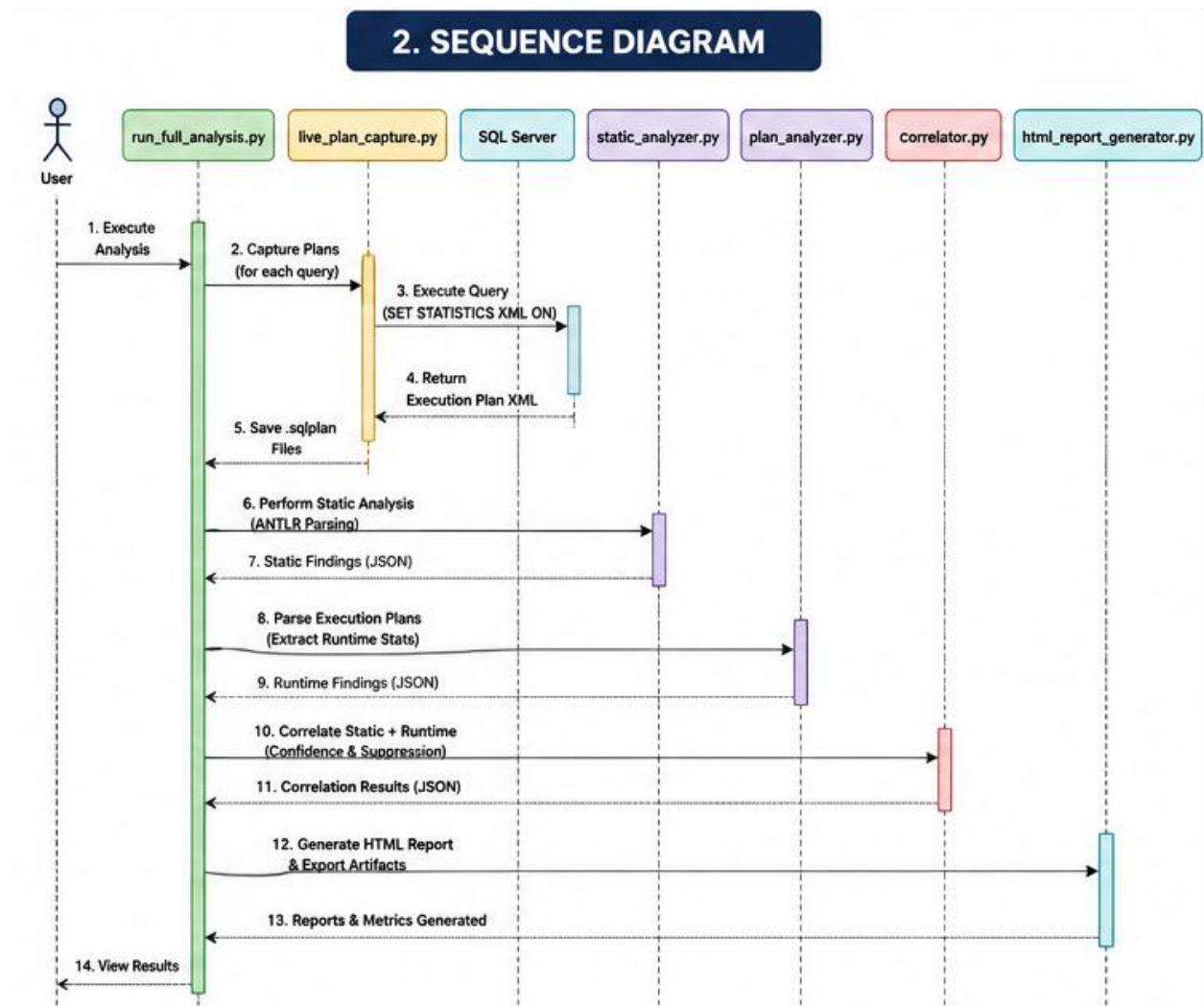


Figure 2: Illustrates the live execution-plan capture workflow between QueryLens and SQL Server.

The sequence diagram shows the execution order of a full QueryLens analysis run. It is useful because QueryLens is not a single-step analyzer; it performs plan capture, static parsing, runtime parsing, correlation, and report generation as separate stages.

In offline mode, QueryLens analyzes existing .sql files and matching .sqlplan files. This mode is useful for reproducible experimentation because the execution plans are already stored and do not require an active SQL Server connection.

In live mode, QueryLens connects to SQL Server through pyodbc, executes read-only workload queries, captures actual execution plans using SET STATISTICS XML ON, and stores the resulting plans in the plans_live/ directory. This mode provides stronger runtime evidence because the captured plans include actual row counts and actual operator execution counts.

2.5.5 Data Flow Diagram

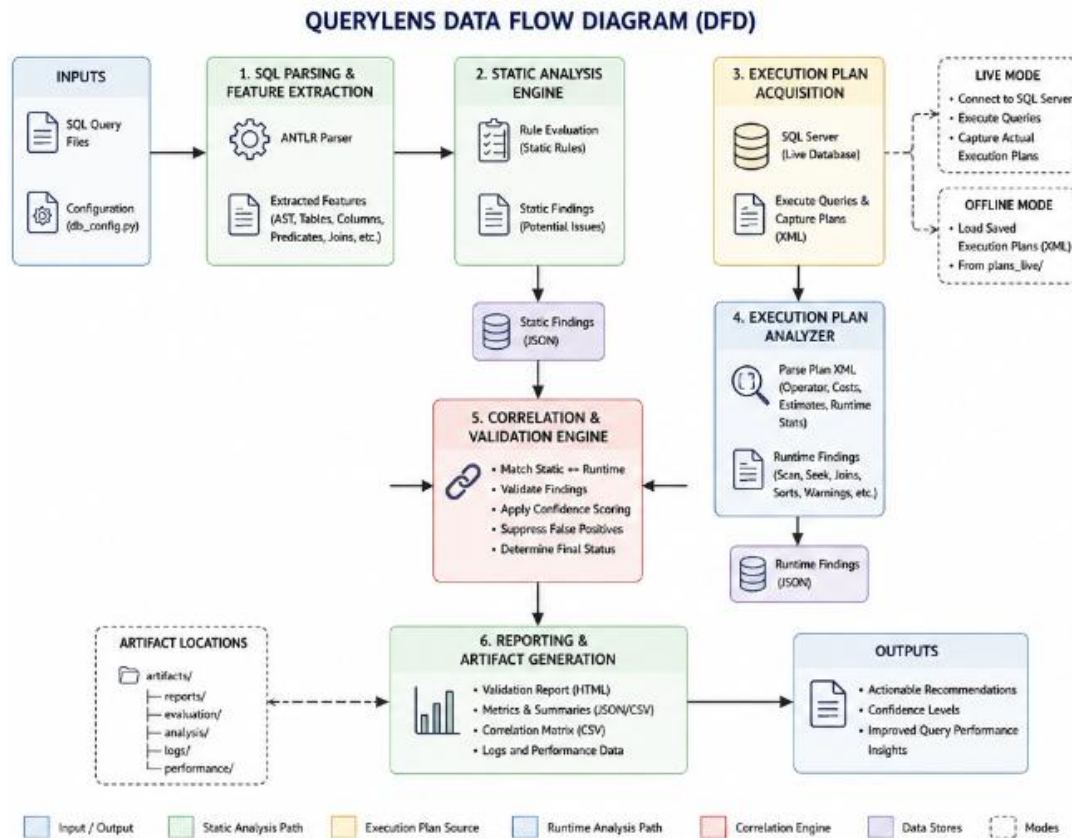


Figure 3: Data Flow Diagram (DFD) of the QueryLens system

The data flow diagram shows how QueryLens transforms raw SQL workload files into validated analysis artifacts. SQL files first enter the parsing and feature extraction stage, producing structured static findings. Execution plans are either loaded from existing .sqlplan files or captured from SQL Server during live execution, then parsed into runtime findings. Both streams flow into the correlation engine, where static findings are compared against runtime evidence. The final outputs are stored as JSON, CSV, HTML, image, and log artifacts that support reporting, evaluation, and reproducibility.

2.5.6 Static Analysis Engine

The static analysis component evaluates extracted features against a set of predefined rules representing common SQL performance anti-patterns. These include:

- Use of SELECT *
- Non-SARGable predicates (e.g., functions on indexed columns)
- ORDER BY operations without supporting indexes
- Complex joins and correlated subqueries
- Window functions and derived tables

Each detected issue is recorded as a structured finding with associated metadata and recommended optimizations. Rule evaluation operates over extracted AST features rather than raw SQL text, improving structural accuracy and making the rule engine easier to extend as new SQL patterns are added.

QueryLens supports multi-rule detection, allowing a single SQL query to trigger multiple performance warnings, each independently validated against runtime evidence.

2.5.7 Execution Plan Analysis

Execution plans generated by SQL Server are parsed to extract runtime characteristics, including:

- Physical operators (e.g., Index Scan, Index Seek, Sort, Hash Match)
- Logical operators
- Estimated row counts
- Actual row counts
- Actual execution counts
- Join strategies and aggregation operations
- Estimated subtree cost

Unlike the initial prototype design, the final version of QueryLens supports both offline execution-plan analysis and live execution-plan capture through direct SQL Server integration using pyodbc. Live execution plans are generated dynamically using SQL Server's SET STATISTICS XML ON functionality, allowing QueryLens to retrieve actual runtime execution metrics directly from query execution.

This execution-plan evidence allows the system to evaluate how queries behave during execution rather than relying solely on static SQL inspection. Runtime features extracted from plans are used to validate static findings, suppress unsupported warnings, and strengthen confidence scoring for confirmed performance issues.

Actual runtime statistics such as actual row counts and actual operator executions significantly improve the reliability of runtime validation because they provide direct evidence of query behavior under execution.

QueryLens also integrates live SQL Server index metadata retrieval through pyodbc. Indexed columns are extracted dynamically from SQL Server system catalog views and incorporated into static rule evaluation. This allows runtime validation and missing-index detection to consider actual database indexing structures rather than relying solely on scan-based heuristics. The system additionally aggregates execution-plan operator statistics such as scan counts, seek counts, sort counts, hash joins, nested loops, and key lookups to improve runtime reporting clarity and diagnostic interpretability.

```
C:\Users\Par\Documents\Old Computer Files\BCIT\1 COMP8047\QueryLens>python -m src.db.index_metadata_loader
Loaded SQL Server index metadata
customers: ['createddate', 'customerid']
orders: ['customerid', 'orderdate', 'orderid']
products: ['category', 'productid']
orderitems: ['orderid', 'orderitemid', 'productid']
```

Figure 4: Live SQL Server index metadata retrieval used for index-aware rule validation.

The index metadata loader dynamically retrieves indexed columns from SQL Server system catalog views and incorporates them into runtime-aware rule evaluation. This runtime metadata integration allows QueryLens to distinguish between indexed and non-indexed access paths during rule evaluation. As a result, static findings related to sorting, filtering, and predicate selectivity can be validated against the actual indexing structure present in SQL Server.

2.5.8 Correlation and Validation Engine

The core innovation of QueryLens lies in its correlation engine, which links static warnings to execution plan evidence. Each rule is classified as either runtime-verifiable or non-verifiable. For verifiable rules, the system evaluates whether execution plan characteristics support or contradict the static finding. This validation-aware correlation layer represents the primary research contribution of QueryLens.

A confidence scoring mechanism assigns levels (high, medium, low, none) based on the strength of runtime evidence. Static findings that lack supporting evidence are suppressed, reducing false positives.

Low-confidence correlations are automatically suppressed when insufficient runtime evidence is present, preventing weak matches from being treated as confirmed performance issues.

2.5.9 Reporting and Artifact Generation

The system produces a comprehensive set of artifacts, including:

- Structured JSON outputs for static and runtime analysis
- Evaluation metrics and correlation matrices
- False positive analysis logs
- Interactive HTML reports summarizing findings, runtime operator statistics, execution evidence, confidence levels, and optimization recommendations
- The final HTML reporting layer includes per-query operator summaries displaying scans, seeks, sorts, hash joins, nested loops, and key lookups extracted from execution plans. This improves interpretability of runtime behavior and strengthens the transparency of validation results.

This layered output structure ensures transparency and facilitates further analysis. The reporting layer also supports reproducibility and experimental traceability. Because each pipeline run generates structured artifacts in consistent locations, results can be reviewed, compared, and regenerated from the same workload. This makes the system useful not only as a diagnostic tool,

but also as an evaluation framework for measuring rule behavior, runtime agreement, false-positive reduction, and reporting consistency.

2.6 Installation Manual

2.6.1 Workstation Requirements

QueryLens was developed and tested on a Windows workstation. The recommended environment is:

Requirement	Recommended Version
Operating System	Windows 10 or 11
Python	Python 3.11.9 or later
Database	SQL Server 2019 Developer Edition
SQL Tools	SQL Server Management Studio
Database Driver	ODBC Driver 17 for SQL Server
Memory	16+ GB RAM
Storage	At least 10GB+
Processor	Modern Multi-core CPU

2.6.2 Python Environment Setup

1. Install Python 3.11.9 or later (<https://www.python.org/downloads/windows/>).
2. Verify Python is available from the command prompt:
`python --version`
3. Navigate to the QueryLens project directory.
4. Install the required Python packages:
`pip install antlr4-python3-runtime`
`pip install pyodbc`
`pip install pandas`
`pip install matplotlib`

Dependencies can also be installed using:
`pip install -r requirements.txt`

2.6.3 SQL Server Configuration

1. Install SQL Server 2019 Developer Edition.
2. Install SQL Server Management Studio.
3. Install ODBC Driver 17 for SQL Server.
4. Create or restore the QueryLensDB database.
5. Ensure the database connection supports either Windows trusted authentication or SQL Server authentication.
6. Confirm connectivity before running the full pipeline.

2.6.4 Dataset Setup

The benchmark schema and seed data are located in the datasets directory. The following scripts should be executed in order:

1. datasets\1_schema.sql
2. datasets\2_seed_data.sql
3. datasets\3_schema_expanded.sql
4. datasets\4_seed_data_expanded.sql

These scripts create and populate the Customers, Orders, Products, and OrderItems tables used by the validation workload.

2.6.5 Configure db_config.py

The database connection settings are stored in src\config\db_config.py. The file should be configured for the local SQL Server instance. A typical trusted-connection configuration is:

```
SERVER = "localhost"
```

```
DATABASE = "QueryLensDB"
```

```
DRIVER = "ODBC Driver 17 for SQL Server"
```

```
TRUSTED_CONNECTION = "yes"
```

Credentials should not be committed to version control.

2.6.6 Running QueryLens

QueryLens supports both offline and live execution-plan analysis modes. The execution mode is controlled through the ENABLE_LIVE_CAPTURE setting in src/config/db_config.py.

When ENABLE_LIVE_CAPTURE = False, QueryLens operates in offline mode and loads existing .sqlplan files from the plans/ directory. This mode does not require a live SQL Server connection and is recommended for reproducible evaluation.

When ENABLE_LIVE_CAPTURE = True, QueryLens connects to SQL Server through pyodbc, captures fresh actual execution plans using SET STATISTICS XML ON, and stores generated plans in the plans_live/ directory.

After the environment and database are configured, the full analysis pipeline can be executed with:

```
python -m src.run_full_analysis
```

The command runs static analysis, execution-plan capture or loading, runtime parsing, correlation, evaluation, and report generation. The main outputs are written to the artifacts/ directory, while live execution plans are written to plans_live/. A full live-mode run over the 30-query workload took approximately 11.4 seconds in the final evaluation environment.

2.7 User Manual

QueryLens is designed to operate as a validation-aware batch analysis system over a workload of SQL files.

To run the complete system, the user executes the main pipeline command:

```
python -m src.run_full_analysis
```

The pipeline processes all SQL workload files and executes the following stages:

1. static SQL analysis
2. feature extraction
3. live or offline execution-plan analysis
4. static/runtime correlation
5. rewrite suggestion generation
6. evaluation artifact generation
7. report generation

2.7.1 Offline Mode Workflow

Offline mode is used when SQL workload files and matching .sqlplan files already exist. This mode is useful for reproducible testing because it does not require a live SQL Server connection during analysis. The user places workload .sql files and matching .sqlplan files in the plans/ directory and runs:

```
python -m src.run_full_analysis
```

QueryLens then parses the SQL files, loads the stored execution plans, performs correlation, and regenerates evaluation artifacts and reports.

2.7.2 Live Mode Workflow

Live mode uses pyodbc to connect directly to SQL Server and capture fresh actual execution plans. In this mode, only the workload .sql files are required. QueryLens validates that each workload query is read-only, executes the query against the validation database, captures the actual execution plan using SET STATISTICS XML ON, and saves the resulting .sqlplan files into plans_live/.

Live mode provides stronger evidence because actual row counts and actual operator execution counts are available for correlation and reporting.

2.7.3 Generated Outputs

Artifact	Location	Purpose
Static_results.json	Artifacts\analysis	Static SQL Findings
Validated_results.json	Artifacts\analysis	Runtime-confirmed findings
Correlation_output.json	Artifacts\analysis	Detailed static/runtime correlation results
Expanded_runtime_report.json	Artifacts\evaluation	Per-query runtime validation summary
Rule_level_metrics.json	Artifacts\evaluation	Rule-level evaluation metrics
Correlation_matrix.csv	Artifacts	Rule confirmation matrix
Performance_log.csv	Artifacts	Pipeline runtime measurements

Validation_log.txt	Artifacts	False-positive and suppression log
Runtime_validation_report.html	Artifacts\reports	Human-readable validation report
.sqlplan files	Plans_live	Captured live execution plans

2.7.4 Common Errors

Error	Likely Cause	Resolution
SQL Server connection failed	Incorrect server/database name or authentication settings	Verify db_config.py and confirm SQL Server is running
ODBC driver not found	ODBC Driver 17 not installed	Install Microsoft ODBC Driver 17 for SQL Server
XML parse error	Execution plan is malformed or not valid XML	Regenerate the .sqlplan file or rerun live capture
Missing execution plan	Offline mode cannot find a matching .sqlplan file	Add the plan file or use live mode
Query rejected	Query contains INSERT, UPDATE, DELETE, DROP, ALTER, Merge, or TRUNCATE	Use only read-only SELECT workload queries

2.8 Testing Details and Results

Testing was performed using a controlled 30-query benchmark workload designed to exercise both static SQL rule detection and runtime execution-plan validation. The workload included intentionally inefficient queries, optimized queries, joins, aggregations, subqueries, window functions, ORDER BY cases, and indexed versus non-indexed access patterns.

Testing was divided into three categories: static analysis testing, runtime validation testing, and reporting validation. Static testing verified that the parser and rule engine detected expected SQL patterns. Runtime validation testing confirmed that SQL Server execution-plan evidence supported or contradicted static findings. Reporting validation ensured that generated artifacts were complete, consistent, and usable for evaluation.

2.8.1 Static Analysis Testing

Static analysis testing focused on whether QueryLens could correctly identify SQL performance patterns from query text. Test cases included SELECT * usage, non-SARGable predicates, implicit conversion candidates, ORDER BY clauses, joins, correlated subqueries, derived tables, HAVING clauses, and window functions.

The parser was validated using controlled workload queries with known expected outcomes. Each query was processed through the ANTLR-based parser and FeatureExtractor, and the resulting feature set was passed into the static rule engine. The generated findings were compared against expected rule outputs and stored in static analysis artifacts such as static_results.json and static_test_log.txt.

2.8.2 Runtime Validation Testing

Runtime validation testing focused on whether execution-plan evidence supported the static findings. QueryLens evaluated scan versus seek behavior, sort operators, hash joins, nested loops, actual row counts, actual execution counts, and missing-index indicators.

Live validation used pyodbc and SET STATISTICS XML ON to capture actual SQL Server execution plans. This allowed the system to compare static detections against observed runtime behavior rather than relying only on estimated plan information. Runtime validation was especially important for suppressing weak findings and confirming high-impact cases such as non-SARGable predicates, SELECT * queries with large scans, and ORDER BY operations requiring explicit sorts.

2.8.3 Reporting Validation

Reporting validation confirmed that QueryLens generated consistent and usable output artifacts. The HTML report was checked for per-query summaries, rule findings, confidence values, recommendations, rewrite suggestions, runtime evidence blocks, and operator summary cards. JSON and CSV outputs were reviewed to confirm that static findings, validated findings, correlation results, and rule-level metrics were written consistently.

Additional validation considered error handling and edge cases, including malformed SQL, missing execution plans, duplicate findings, static-only advisory rules, and runtime fallback behavior when actual execution statistics were unavailable.

2.9 Implications of Implementation

The implementation of QueryLens has practical implications for SQL Server development and support workflows. In production systems, query performance problems often appear as user-facing slowdowns, delayed reports, or support incidents. These problems can be difficult to diagnose quickly because developers may see the SQL text while database administrators rely on execution plans, creating a gap between code-level understanding and runtime behavior.

QueryLens helps reduce this gap by translating execution-plan evidence into development-oriented feedback. This supports more proactive optimization because inefficient patterns can be detected and validated before they become production bottlenecks. The system can also reduce troubleshooting effort by prioritizing findings that are supported by runtime evidence rather than overwhelming users with every possible static warning.

The project is especially relevant for teams without dedicated database tuning specialists. By combining static analysis, execution-plan interpretation, confidence scoring, and reporting into one workflow, QueryLens makes SQL performance analysis more accessible to developers, analysts, and support staff.

2.10 Innovation

The main innovation of QueryLens is its validation-aware architecture. Many SQL analysis approaches focus either on static inspection or on execution-plan review. QueryLens combines both approaches by detecting SQL anti-patterns in the source query and then validating those findings against SQL Server execution-plan evidence.

This design allows QueryLens to reduce false positives by suppressing weak or unsupported findings. Instead of treating every detected pattern as a confirmed problem, the system evaluates runtime operators, actual row counts, actual executions, index metadata, and confidence thresholds before presenting recommendations. This makes the output more trustworthy than static-only warnings.

Another innovative aspect is the integration of live SQL Server execution-plan capture and index metadata retrieval through pyodbc. This allows the system to incorporate actual runtime behavior and actual database indexing structures into the analysis. As a result, QueryLens functions as more than a SQL linter or plan viewer; it acts as a correlation layer between SQL code, runtime behavior, and user-facing recommendations.

2.11 Complexity

QueryLens meets the complexity expectations of an advanced academic software project because it integrates several technically challenging components into a single reproducible system. The static analysis layer required ANTLR grammar integration, abstract syntax tree traversal, feature extraction, and rule evaluation. This is more complex than simple string matching because the system must interpret SQL structure rather than only search for keywords.

The runtime analysis layer required parsing SQL Server XML execution plans, extracting physical and logical operators, handling actual and estimated row counts, identifying execution patterns, and preserving fallback behavior when actual runtime statistics were unavailable. Live mode added further complexity by requiring pyodbc integration, safe query execution controls, automatic execution-plan capture, and plan storage.

The correlation layer added another level of difficulty because static findings and runtime evidence do not map one-to-one. The system therefore required rule normalization, runtime feature aggregation, confidence scoring, suppression thresholds, static-only classification, and artifact generation. The result is a multi-stage pipeline that supports both offline and live analysis while producing consistent JSON, CSV, graphical, and HTML outputs.

2.12 Research in New Technologies

QueryLens incorporates several technologies that were selected specifically to support accurate, reproducible SQL performance analysis.

The first key technology is ANTLR, which was used to support grammar-aware parsing of T-SQL queries. Rather than relying only on regular-expression heuristics, the use of ANTLR allowed QueryLens to parse SQL into abstract syntax trees and inspect structural features such as joins, subqueries, HAVING clauses, window functions, and derived tables. This provided greater precision and extensibility in rule detection.

The second key technology is SQL Server execution plan analysis. QueryLens parses SQL Server XML plan files to extract runtime evidence including scans, seeks, joins, sorts, parallelism operators, estimated row counts, actual row counts, and actual execution counts. This allows the system to

move beyond purely theoretical warnings and instead assess whether static findings are supported by observed execution behavior.

The third important technology is pyodbc, which provides direct SQL Server connectivity. QueryLens uses pyodbc to establish live database connections, execute workload queries safely, and dynamically retrieve actual execution plans through SQL Server's SET STATISTICS XML ON functionality. This integration enables the system to automatically generate execution plans and collect actual runtime statistics without requiring manual plan export.

QueryLens additionally uses pyodbc to retrieve SQL Server index metadata directly from system catalog views. This metadata is incorporated into static rule evaluation to improve missing-index detection and reduce over-triggering of index-related warnings.

The fourth important technical element is the system's validation-aware correlation engine, which links code-level detections to execution-plan evidence. This correlation layer represents the main research and implementation contribution of the project because it enables selective confirmation, suppression, and confidence scoring of findings.

Finally, QueryLens uses structured artifact generation and reporting to ensure reproducibility. JSON, CSV, and HTML outputs are produced from the same core pipeline, making the system suitable both for experimentation and for end-user diagnostic reporting.

Together, these technologies support the project's goal of producing a practical and evidence-based SQL performance analysis tool rather than a static linter or a plan viewer alone.

2.13 Future Enhancements

Future enhancements could include:

- Expanding the rule set to cover additional SQL patterns
- Incorporating cost-based analysis using execution plan metrics
- Extending support to other database systems
- Integrating machine learning techniques for adaptive rule weighting
- Supporting composite-index analysis and covering-index evaluation
- Extending runtime validation to additional execution-plan operators
- Automatic stored procedure extraction

2.14 Timeline and Milestones

The development of QueryLens followed an incremental milestone-based schedule that allowed core components to be implemented and validated progressively.

The first phase focused on **environment setup, execution plan parsing, and dataset preparation**. This established the technical foundation of the project by configuring SQL Server, Python tooling, and the initial plan-analysis prototype. It also produced the baseline workload and seeded datasets required for later evaluation.

The second phase focused on **static SQL analysis and parser integration**. During this stage, the original heuristic analysis was refactored into a grammar-aware pipeline using ANTLR, and automated testing was introduced to validate rule detection.

The third phase emphasized **runtime validation and evaluation design**. This included development of the static/runtime correlation engine, generation of correlation metrics, false-positive diagnostics, and HTML reporting.

The fourth phase extended the project through **expanded workload evaluation, rule refinement, and architectural consolidation**. The benchmark workload was increased to 30+ queries, runtime confirmation logic was refined, and the evaluation workflow was standardized around the final validation-aware pipeline.

The final phase focused on **recommendation generation, proposal traceability, and submission readiness**. During this stage, rewrite suggestions were integrated, proposal-supporting artifacts were regenerated from the main pipeline, and the repository structure, documentation, and reporting outputs were finalized.

This milestone structure made it possible to progress from a narrow prototype into a fully integrated and reproducible analysis system while continuously validating intermediate results.

The development timeline is further supported by version-controlled commit history, where each milestone corresponds to a recorded weekly commit documenting implemented features, evaluation changes, and system refinements.

3. Implementation

QueryLens is implemented in Python and follows a modular architecture to ensure maintainability and extensibility.

3.1 Static Analysis Implementation

Static analysis is performed using an AST-based approach rather than regular expressions, improving detection accuracy and reducing ambiguity. It also incorporates optional live SQL Server index metadata into static rule evaluation. Extracted metadata is used to validate ORDER BY and join-related heuristics against actual indexed columns in the database schema, improving realism and reducing false-positive index recommendations.

The implementation uses a FeatureExtractor class based on ANTLR visitor traversal. The extractor walks the parse tree and records structural SQL features such as SELECT * usage, WHERE clauses, joins, ORDER BY columns, subqueries, HAVING clauses, and window functions. These extracted features are then passed to the static rule engine, which produces normalized findings used later by the correlation engine.

3.2 Runtime Analysis Implementation

Runtime analysis is performed through a combination of execution-plan parsing and live SQL Server integration.

QueryLens supports both offline execution-plan analysis using stored .sqlplan files and live execution-plan capture through pyodbc database connectivity. In live mode, workload queries are executed directly against SQL Server using a controlled validation dataset. Execution plans are captured automatically using SQL Server's SET STATISTICS XML ON functionality.

To ensure safe execution behavior, QueryLens restricts live execution to read-only SELECT-style workload queries. Queries containing potentially destructive operations such as INSERT, UPDATE, DELETE, DROP, ALTER, MERGE, or TRUNCATE statements are automatically rejected.

Captured execution plans are parsed to extract operator-level runtime details including:

- scans
- seeks
- sorts
- joins
- hash aggregations
- estimated row counts
- actual row counts
- actual operator executions
- estimated subtree costs

Actual runtime statistics are used preferentially when available, while estimated metrics are retained as fallback behavior for offline execution plans that lack runtime counters.

The system dynamically generates .sqlplan files during execution and stores them in a dedicated plans_live/ directory. This allows the complete runtime-validation workflow to be reproduced from workload SQL files alone.

The runtime analysis layer also aggregates operator-level execution statistics including scan counts, seek counts, sort counts, hash joins, nested loops, and key lookups. These summaries are integrated into the reporting layer to improve visualization and interpretation of execution-plan behavior.

When actual runtime statistics are unavailable, QueryLens automatically falls back to estimated execution-plan metrics to preserve compatibility with offline execution-plan analysis.

3.2.1 Live Execution Plan Capture

A significant enhancement introduced during the later stages of development was live execution-plan capture through direct SQL Server integration.

QueryLens uses pyodbc to establish a live SQL Server connection and execute workload queries against the validation dataset. Before execution, the system validates queries to ensure that only read-only SELECT-style workloads are permitted. This prevents accidental execution of destructive SQL statements.

```
C:\Users\Par\Documents\Old Computer Files\BCIT\1 COMP8047\QueryLens>python -m src.db.db_connection
Connected to QueryLensDB
SELECT 1 succeeded
```

Figure 5: Successful live SQL Server connectivity test using pyodbc and QueryLensDB.

This connectivity validation confirmed that QueryLens could establish a live SQL Server session and execute controlled validation queries before runtime plan capture.

Execution plans are retrieved dynamically using SQL Server's SET STATISTICS XML ON functionality. After query execution completes, QueryLens extracts the generated XML execution plan and stores it automatically in the plans_live/ directory.

Unlike estimated execution plans alone, live execution plans include runtime execution statistics such as actual row counts and actual operator execution counts. These runtime statistics significantly improve the quality of validation-aware correlation because the system can evaluate observed execution behavior rather than relying solely on estimated optimizer predictions.

This enhancement removed the earlier requirement for users to manually export execution plans and substantially improved the automation and practicality of the QueryLens workflow.

3.3 Correlation Engine Implementation

The correlation engine evaluates each static finding against runtime evidence. For example:

- A SELECT * warning is confirmed if scans and large row counts are present
- A missing WHERE warning may be suppressed if runtime evidence does not indicate inefficiency
- False positive logs demonstrate that several static warnings were correctly suppressed due to insufficient runtime evidence, improving overall accuracy.

Rewrite suggestions are generated only after correlation analysis, allowing QueryLens to prioritize recommendations supported by execution-plan evidence rather than static detection alone.

The implementation additionally incorporates actual execution counts and runtime operator summaries into correlation evidence payloads. This allows runtime-confirmed findings to expose both execution frequency and operator distribution directly within generated validation reports.

4. Evaluation

The system was evaluated using a controlled experimental methodology designed to assess the effectiveness of validation-aware SQL analysis. A workload of 30 SQL queries was constructed to represent a range of realistic database scenarios, including joins, aggregations, subqueries, and indexing patterns. For each query, a corresponding SQL Server execution plan was generated and analyzed alongside static detection results.

Testing was performed using a reproducible batch pipeline that applies static analysis, execution plan parsing, and correlation-based validation across the entire workload. Evaluation metrics were derived by comparing static warnings against runtime evidence to measure runtime agreement rates, false positive reduction, and precision improvement. All experiments were executed using stored inputs and execution plans, ensuring consistency and repeatability of results.

The evaluation process produced structured artifacts including validation reports, correlation matrices, performance logs, and comparison summaries, which were used to support analysis and ensure traceability of results.

4.1 Evaluation Environment

Evaluation was performed on a Windows-based local development environment using Microsoft SQL Server 2019 Developer Edition and Python 3.11.9. QueryLens was executed using live SQL Server integration through pyodbc with ODBC Driver 17 for SQL Server. Testing used a controlled synthetic validation dataset containing Customers, Orders, Products, and OrderItems tables populated with high-volume benchmark data. All experiments were executed on a single workstation to ensure reproducibility and consistency across evaluation runs.

4.2 Overall Performance

All evaluation runs were performed using the same workload dataset, validation schema, and automated batch-analysis pipeline. Because QueryLens generates execution plans, correlation artifacts, and evaluation reports programmatically, experimental results can be reproduced consistently across repeated executions under the same environment configuration.

The analysis produced 53 static findings, of which 33 were runtime-verifiable. Among these, 32 findings were supported by execution-plan evidence, resulting in a runtime agreement rate of 97%.

The final validation-aware pipeline also produced:

- 31 high-confidence confirmations
- 1 suppressed finding
- 118 runtime operators extracted from execution plans
- approximately 11.4 seconds total batch runtime during live execution-plan capture mode

Metric	Result
Total Static findings	53
Runtime-Verifiable Findings	33
Runtime-Confirmed Findings	32
Runtime Agreement Rate	97%
High-Confidence Confirmations	31
Suppressed Findings	1
Runtime Operators Extracted	118
False Positive Reduction	65%
Final Combined Precision	64.9%
Live Batch Runtime	~11.4 seconds

These results demonstrate that combining static SQL analysis with runtime validation substantially improves diagnostic precision while maintaining reproducible evaluation behavior.

```
C:\Users\Par\Documents\Old Computer Files\BCIT\1 COMP8047\QueryLens>python -m src.run_full_analysis

=== Running QueryLens Batch Analysis ===

=== Live capture enabled ===
Generating fresh actual execution plans using pyodbc...
Captured actual plan: Q01_SelectStar_Customers_FullScan.sqlplan
Captured actual plan: Q02_Sargable_DateRange_Customers_Seek.sqlplan
Captured actual plan: Q03_NonSargable_DateFunction_Customers_Scan.sqlplan
Captured actual plan: Q04_FunctionPredicate_Customers_Scan.sqlplan
Captured actual plan: Q05_OrderBy_IndexedColumn_Orders.sqlplan
Captured actual plan: Q06_OrderBy_NonIndexed_Customers_Sort.sqlplan
Captured actual plan: Q07_CustomersOrders_BasicJoin.sqlplan
Captured actual plan: Q08_CustomersOrders_AggregateJoin.sqlplan
Captured actual plan: Q09_Join_WithSelectiveFilter.sqlplan
Captured actual plan: Q10_FullSchema_MultiJoin.sqlplan
Captured actual plan: Q11_GlobalAggregation_Orders.sqlplan
Captured actual plan: Q12_GroupBy_ProductCategory.sqlplan
Captured actual plan: Q13_HighSelectivity_OrderLookup.sqlplan
Captured actual plan: Q14_LowSelectivity_DateRange.sqlplan
Captured actual plan: Q15_Join_WithExplicitSort.sqlplan
Captured actual plan: Q16_FilteredAggregation_CustomerOrders.sqlplan
Captured actual plan: Q17_ProductPrice_FilterJoin.sqlplan
Captured actual plan: Q18_ComputedAggregation_Revenue.sqlplan
Captured actual plan: Q19_ProductJoin_NonIndexedSort.sqlplan
Captured actual plan: Q20_WideSchema_SelectStar.sqlplan
Captured actual plan: Q21_Exists.sqlplan
Captured actual plan: Q22_Not_Exists.sqlplan
Captured actual plan: Q23_Left_Join.sqlplan
Captured actual plan: Q24_Cross_Join.sqlplan
Captured actual plan: Q25_Group_By.sqlplan
Captured actual plan: Q26_Top_Order_By.sqlplan
Captured actual plan: Q27_Derived_Table.sqlplan
Captured actual plan: Q28_Window.sqlplan
Captured actual plan: Q29_Multi_Condition_Where.sqlplan
Captured actual plan: Q30_Correlated_Subquery.sqlplan

Live capture log saved → C:\Users\Par\Documents\Old Computer Files\BCIT\1 COMP8047\QueryLens\artifacts\live_capture_log.txt
Live execution-plan capture complete

=== Loading SQL Server index metadata ===
Index metadata loaded
→ Processing: Q01_SelectStar_Customers_FullScan
→ Processing: Q02_Sargable_DateRange_Customers_Seek
→ Processing: Q03_NonSargable_DateFunction_Customers_Scan
→ Processing: Q04_FunctionPredicate_Customers_Scan
→ Processing: Q05_OrderBy_IndexedColumn_Orders
→ Processing: Q06_OrderBy_NonIndexed_Customers_Sort
→ Processing: Q07_CustomersOrders_BasicJoin
→ Processing: Q08_CustomersOrders_AggregateJoin
→ Processing: Q09_Join_WithSelectiveFilter
→ Processing: Q10_FullSchema_MultiJoin
→ Processing: Q11_GlobalAggregation_Orders
→ Processing: Q12_GroupBy_ProductCategory
```

```

→ Processing: Q12_GroupBy_ProductCategory
→ Processing: Q13_HighSelectivity_OrderLookup
→ Processing: Q14_LowSelectivity_DateRange
→ Processing: Q15_Join_WithExplicitSort
→ Processing: Q16_FilteredAggregation_CustomerOrders
→ Processing: Q17_ProductPrice_FilterJoin
→ Processing: Q18_ComputedAggregation_Revenue
→ Processing: Q19_ProductJoin_NonIndexedSort
→ Processing: Q20_WideSchema_SelectStar
→ Processing: Q21_Exists
→ Processing: Q22_Not_Exists
→ Processing: Q23_Left_Join
→ Processing: Q24_Cross_Join
→ Processing: Q25_Group_By
→ Processing: Q26_Top_Order_By
→ Processing: Q27_Derived_Table
→ Processing: Q28_Window
→ Processing: Q29_Multi_Condition_Where
→ Processing: Q30_Correlated_Subquery
✓Validation-aware metrics saved
✓Expanded runtime validation complete
✓Rule-level metrics generated: C:\Users\Par\Documents\Old Computer Files\BCIT\1 COMP8047\QueryLens\artifacts\evaluation\rule_level_metrics.json
✓HTML report generated: C:\Users\Par\Documents\Old Computer Files\BCIT\1 COMP8047\QueryLens\artifacts\reports\runtime_validation_report.html
✓Static test log generated: C:\Users\Par\Documents\Old Computer Files\BCIT\1 COMP8047\QueryLens\artifacts\static_test_log.txt
Correlation matrix saved
→ C:\Users\Par\Documents\Old Computer Files\BCIT\1 COMP8047\QueryLens\artifacts\correlation_matrix.csv
✓False positive analysis complete
Static accuracy evaluation complete
{
  "total_true_positive": 25,
  "total_false_positive": 0,
  "total_false_negative": 0,
  "precision": 1.0,
  "recall": 1.0,
  "f1_score": 1.0
}
Comparison summary saved → C:\Users\Par\Documents\Old Computer Files\BCIT\1 COMP8047\QueryLens\artifacts\comparison_summary.csv
Combined chart saved → C:\Users\Par\Documents\Old Computer Files\BCIT\1 COMP8047\QueryLens\artifacts\combined_vs_baseline.png
Performance log generated → C:\Users\Par\Documents\Old Computer Files\BCIT\1 COMP8047\QueryLens\artifacts\performance_log.csv
Batch analysis complete
Global evaluation metrics generated
Expanded runtime evaluation generated
Rule-level metrics generated
HTML report generated
Static test log generated
Correlation matrix generated
False positive validation log generated
Baseline vs combined comparison artifacts generated

Summary:
Total queries processed: 30
Total static findings: 53
Total runtime operators: 118
Total correlated findings: 53
Validated findings: 33
Static-only findings: 20
Runtime-supported findings: 32
Runtime agreement rate: 0.97
Suppressed findings: 1
Suppression rate: 0.03
High-confidence confirmations: 31
Total pipeline runtime (seconds): 11.0071

```

Figure 6: Full QueryLens live analysis pipeline execution showing runtime plan capture, validation processing, artifact generation, and evaluation metrics.

The live pipeline execution demonstrates successful end-to-end integration between static analysis, live SQL Server execution-plan capture, runtime validation, artifact generation, and reporting.

QueryLens Validation-Aware Analysis Report

This report summarizes runtime validation results for QueryLens by comparing runtime-verifiable static warnings against SQL Server execution plan evidence.

It includes global metrics, per-query summaries, detailed rule findings, plain-language recommendations, and rewrite suggestions.

Global Metrics

Total Runtime-Verifiable Static Warnings: 33

Total Runtime-Confirmed Findings: 32

Confirmation Rate: 0.97

Figure 7: QueryLens runtime validation report showing per-query analysis results, validation status, and confidence levels.

4.3 False Positive Reduction

Static-only analysis generated a large number of false positives. By incorporating runtime validation, QueryLens reduced false positives from 57 to 20, representing a 65% reduction. This demonstrates the effectiveness of combining static and runtime analysis.

4.4 Precision Improvement

The precision of the analysis improved significantly when runtime validation was applied. Static-only analysis resulted in **low effective precision in practice**, highlighting the limitations of unvalidated static detection, whereas the combined approach achieved 64.9% precision, representing a substantial improvement in recommendation quality.

Static-only precision was low because static detection identifies potential risks without knowing whether SQL Server actually executed the query inefficiently. Runtime validation improves precision by filtering unsupported warnings and emphasizing findings confirmed by execution-plan behavior.

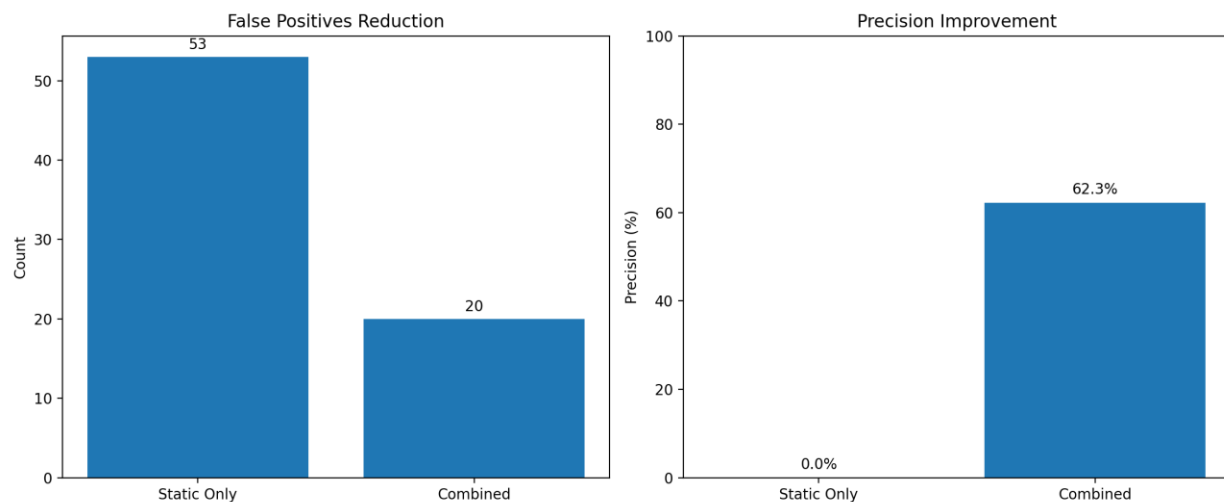


Figure 8: Comparison of static-only analysis and validation-aware QueryLens analysis, illustrating reduction in false positives and improved precision

4.5 Ground Truth Validation

When evaluated against a labeled dataset, QueryLens achieved exact performance on the labeled dataset:

- Precision: 1.0
- Recall: 1.0
- F1 Score: 1.0

The perfect precision and recall scores were achieved because the evaluation workload was intentionally designed as a controlled benchmark suite with known expected outcomes for each rule category. The evaluation was intended to validate the correctness of the correlation pipeline under reproducible conditions rather than benchmark behavior against uncontrolled production workloads.

4.6 Rule-Level Performance

Certain rules demonstrated particularly strong performance:

- non_sargable_predicate: 100% precision
- order_by_no_index: 100% precision
- select_star: 90% precision

rule	total	confirmed	confirmation_rate	with_actual_stats	actual_stats_coverage
select_star	10	9	0.9	10	1
missing_where	7	0	0	7	1
non_sargable_predicate	2	2	1	2	1
order_by_no_index	2	2	1	2	1
basic_join	11	0	0	11	1
missing_index	12	12	1	12	1
complex_join	2	2	1	2	1
exists_subquery	1	1	1	1	1
not_exists_subquery	1	1	1	1	1
cross_join	1	1	1	1	1
having_clause	1	1	1	1	1
derived_table	1	0	0	1	1
window_function	1	1	1	1	1
correlated_subquery	1	0	0	1	1

Figure 9: Rule-level correlation matrix summarizing runtime agreement rates between static detections and execution plan evidence.

In contrast, rules such as missing_where and basic_join were not runtime-verifiable and therefore required suppression logic to prevent misleading recommendations.

Rules such as missing_where, basic_join, and derived_table are weaker runtime indicators because they describe query structure rather than a specific inefficient execution behavior. For this reason, these findings are treated as static-only advisory rules or suppressed when runtime evidence is insufficient. This prevents structural observations from being incorrectly presented as confirmed runtime performance issues.

The missing_index rule was refined during the final implementation phase by incorporating live SQL Server index metadata into rule evaluation. This reduced reliance on broad scan-based heuristics and improved alignment between static warnings and actual database indexing structures.

4.7 Validation Examples

To demonstrate practical validation behavior, QueryLens was evaluated against both optimized and intentionally inefficient SQL queries.

Good-query examples included SARGable indexed filters that produced no unnecessary warnings or rewrite suggestions. For example:

```
SELECT CustomerID FROM Customers WHERE CreatedDate >= '2024-01-01';
```

This query generated no major recommendations because runtime evidence confirmed efficient index-lookup behavior.

▼ Q02_Sargable_DateRange_Customers_Seek.sql

Summary: 0 / 0 runtime-verifiable findings confirmed

Runtime-Verifiable Warnings: 0 | Runtime-Confirmed Findings: 0 | Rate: 0

No runtime-verifiable findings.

This query triggered no runtime-verifiable warnings.

Recommendations

None

Rewrite Suggestions

None

Bad-query examples included patterns such as `SELECT *`, non-SARGable predicates, and non-indexed `ORDER BY` operations.

For example:

```
SELECT * FROM Customers;
```

This query generated a confirmed `select_star` warning with high confidence due to scan evidence and large runtime row counts.

▼ Q01_SelectStar_Customers_FullScan.sql

Summary: 1 / 1 runtime-verifiable findings confirmed

Runtime-Verifiable Warnings: 1 | Runtime-Confirmed Findings: 1 | Rate: 1.0

Operator Summary

Scans 1	Seeks 0	Sorts 0	Hash Joins 0	Nested Loops 0	Key Lookups 0
------------	------------	------------	-----------------	-------------------	------------------

Rule	Confirmed	Confidence	Validation Type	Reason	Recommendation
select_star	True	high	runtime	Scan suggests broad row retrieval; High actual row count increases SELECT * impact; Actual runtime row evidence supports impact; max actual rows = 50000.0 Max Actual Rows: 50000.0 Max Actual Executions: 1.0 Total Actual Executions: 1.0 Scan Count: 1 Actual Stats Available: True	Issue: SELECT * used Impact: May cause unnecessary I/O and prevent index usage Recommendation: Select only required columns instead of using SELECT *
missing_where	N/A	N/A	static-only	Advisory static rule; runtime confirmation not applicable	Issue: No WHERE clause Impact: Full table scan possible Recommendation: Add filtering conditions

Rewrite Suggestions

Rule: select_star

Issue: Using SELECT *

Suggestion: Replace SELECT * with explicit column names

Example:

```
SELECT col1, col2 FROM table_name
```

Similarly:

SELECT CustomerID FROM Customers WHERE YEAR(CreatedDate) = 2025;

▼ Q03_NonSargable_DateFunction_Customers_Scan.sql

Summary: 1 / 1 runtime-verifiable findings confirmed

Runtime-Verifiable Warnings: 1 | Runtime-Confirmed Findings: 1 | Rate: 1.0

Operator Summary

Scans 1	Seeks 0	Sorts 0	Hash Joins 0	Nested Loops 0	Key Lookups 0
------------	------------	------------	-----------------	-------------------	------------------

Rule	Confirmed	Confidence	Validation Type	Reason	Recommendation
non_sargable_predicate	True	high	runtime	Scan detected, which is expected for a non-sargable predicate; High actual row count amplifies non-sargable predicate impact; Actual runtime row evidence supports impact; max actual rows = 46302.0 Max Actual Rows: 46302.0 Max Actual Executions: 1.0 Total Actual Executions: 1.0 Scan Count: 1 Actual Stats Available: True	Issue: Non-SARGable predicate Impact: Prevents index seeks and forces scans Recommendation: Avoid functions on indexed columns

Rewrite Suggestions

Rule: non_sargable_predicate

Issue: Function applied to indexed column

Suggestion: Rewrite predicate to be sargable

Example:

```
WHERE OrderDate >= '2023-01-01' AND OrderDate < '2024-01-01'
```

generated a confirmed non_sargable_predicate finding supported by scan behavior in the execution plan.

These examples demonstrate that QueryLens distinguishes between acceptable SQL patterns and performance risks rather than generating recommendations indiscriminately.

5. Discussion

The results demonstrate that integrating runtime validation into static SQL analysis significantly improves the quality of performance recommendations.

5.1 Strengths

The primary strength of QueryLens lies in its ability to reduce false positives while maintaining high detection accuracy. By grounding static findings in execution-plan evidence, the system provides more reliable and actionable insights.

A major strength of the final implementation is the integration of live SQL Server connectivity through pyodbc. This allows QueryLens to automatically capture execution plans, retrieve actual runtime statistics, and validate findings using real execution behavior rather than relying entirely on estimated execution metrics.

The system also demonstrates strong reproducibility through its structured artifact-generation pipeline, which automatically produces JSON, CSV, graphical, and HTML outputs from a single analysis workflow.

A key contribution of QueryLens is that unsupported static warnings are actively suppressed rather than merely reported, reducing noise and improving recommendation trustworthiness.

Another major strength of the implementation is the use of index-aware validation logic. Rather than treating scans alone as evidence of missing indexes, QueryLens evaluates join and filter columns against actual indexed columns retrieved from SQL Server metadata. This substantially improves realism and reduces over-triggering of missing-index warnings.

Finally, the modular architecture supports extensibility by separating parsing, static analysis, runtime analysis, correlation, reporting, and evaluation into independent components.

5.2 Limitations

Despite its strengths, the system has several limitations related to implementation scope and evaluation constraints.

First, QueryLens currently supports only Microsoft SQL Server. The parser, execution-plan analysis logic, and runtime validation workflow are tightly coupled to SQL Server execution-plan semantics and SQL Server XML plan formats. Supporting additional database systems such as PostgreSQL or MySQL would require substantial adaptation of both parsing and runtime-analysis components.

Second, while QueryLens supports live execution-plan capture and actual runtime statistics, the runtime-validation logic currently focuses on a selected subset of execution-plan operators and performance indicators. Additional operator coverage and deeper cost-based analysis could improve the granularity of runtime validation.

Third, the evaluation methodology focuses on agreement between static analysis and execution-plan evidence rather than absolute ground-truth correctness. Comprehensive ground-truth labeling

for all possible SQL inefficiencies is not feasible, and therefore the system evaluates consistency and runtime confirmation rather than universal correctness.

Fourth, the evaluation dataset is synthetic, although designed to reflect realistic SQL Server workloads. While it captures a wide range of query patterns, it does not fully represent the complexity and variability of large-scale production systems.

Although QueryLens now incorporates live index metadata into rule evaluation, the current implementation evaluates only single-column index presence and does not yet fully analyze composite index ordering, index selectivity, or covering-index suitability.

Finally, the current implementation primarily evaluates standalone SQL workloads. Although the parser and analysis pipeline operate on T-SQL text and can therefore analyze stored procedure bodies, the system does not currently perform automatic database introspection or stored procedure extraction directly from SQL Server metadata.

These limitations reflect deliberate design trade-offs made to ensure feasibility, reproducibility, and controlled evaluation within the scope of the project.

6. Conclusion

QueryLens demonstrates that combining static SQL analysis with execution plan validation provides a practical and effective approach to SQL query performance optimization. The system successfully reduces false positives, improves diagnostic precision, and delivers more reliable recommendations compared to static analysis alone.

By bridging the gap between code-level analysis and runtime execution behavior, QueryLens provides a valuable tool for developers and database administrators seeking to improve SQL performance. The results of this project demonstrate that validation-aware analysis is a promising direction for future research and development in database performance tools.

The final implementation further improved realism and diagnostic precision through integration of live SQL Server index metadata, enabling QueryLens to evaluate performance risks against actual database indexing structures. Additional support for live execution-plan capture, runtime operator analysis, and actual execution statistics strengthened the system's ability to validate findings using observed execution behavior rather than theoretical analysis alone.

The primary contribution of this work is demonstrating that validation-aware correlation between static SQL analysis and execution-plan evidence can substantially improve diagnostic precision while reducing unsupported performance recommendations.

Overall, QueryLens demonstrates a practical, scalable, and reproducible approach to SQL performance analysis that can be extended to real-world development and database administration environments. The same validation-aware approach could be extended in future database tooling to support broader SQL workloads, additional database systems, automated stored procedure extraction, and more advanced cost-based optimization guidance.

7. Professional and Academic Development

This major project significantly strengthened expertise in the database specialization by connecting theoretical concepts from coursework to the design of a practical software system. The project required applied use of SQL optimization principles, execution plan interpretation, parser design, rule-based analysis, evaluation methodology, and structured reporting. Through QueryLens, concepts such as non-SARGability, indexing strategy, join selectivity, and runtime operator behavior were translated from classroom topics into an implemented tool with measurable results.

The project also extended skills beyond traditional coursework by requiring the integration of multiple technical domains into a single end-to-end system. This included grammar-based parsing with ANTLR, structured artifact generation, experimental benchmarking, and evaluation of detection quality using both labeled datasets and runtime validation evidence. As a result, the project strengthened both technical specialization and systems-level design thinking.

8. Lessons Learned

Several lessons were learned during the development of QueryLens beyond what is typically covered in classroom exercises.

The first lesson was that **accurate diagnostics require more than correct detection logic**. Early static analysis components could successfully identify common SQL anti-patterns, but without runtime context they also produced unnecessary warnings. This reinforced the importance of evidence-based validation rather than relying solely on heuristic pattern matching.

The second lesson was that **software architecture matters as much as algorithm design**. As the project expanded, modularization, artifact organization, and pipeline consolidation became essential to keeping the system maintainable and reproducible. Refactoring and standardization were therefore not secondary cleanup tasks, but core parts of completing the project successfully.

The third lesson was that **evaluation design is itself a major engineering task**. Building a reproducible benchmark workload, generating comparable execution plans, constructing ground-truth datasets, and producing report-ready artifacts required substantial effort and careful methodological choices. This experience demonstrated that high-quality software projects depend not only on implementation, but also on the quality and transparency of their validation process.

Finally, the project reinforced the practical value of building systems around real operational problems. By grounding the work in realistic SQL Server performance issues, QueryLens became more than an academic prototype; it became a focused demonstration of how database expertise, software engineering, and evaluation discipline can be combined into a useful diagnostic tool.

Appendix — Proposal Changes

The core objectives of the project remained consistent with the approved proposal. However, several implementation refinements were introduced during development, including artifact restructuring, evaluation pipeline consolidation, and expansion of proposal-supporting outputs to improve reproducibility and evaluation transparency.

The original project proposal defined a set of core artifacts to support static analysis, execution plan parsing, and report generation. During implementation, these artifacts were refined and expanded to support validation-aware evaluation and reproducibility.

Several proposed artifacts were reorganized into a structured directory hierarchy to improve clarity and maintainability. For example, analysis outputs such as `static_results.json` and `plan_results.json` were moved into a dedicated `artifacts/analysis/` directory, while evaluation outputs were grouped under `artifacts/evaluation/`.

In addition to restructuring, new artifacts were introduced to support the validation-aware design of QueryLens. These include correlation matrices, false positive validation logs, performance benchmarking outputs, and comparison artifacts demonstrating improvements over static-only analysis. These additions were necessary to provide empirical evidence supporting the project's primary objective of reducing false positives and improving diagnostic precision.

The final system therefore extends the original proposal by not only generating diagnostic outputs but also producing a comprehensive set of evaluation artifacts that enable reproducible experimentation and quantitative analysis. These enhancements do not change the core objectives of the project but strengthen the evaluation methodology and improve the transparency of results.

References

ANTLR. *ANTLR Documentation*.

ANTLR. *grammars-v4: T-SQL Grammar*. GitHub Repository.

Fritchey, Grant. *SQL Server Execution Plans*. Redgate Publishing, 2012.

Microsoft. *Display an Actual Execution Plan*. Microsoft Learn.

Microsoft. *Query Processing Architecture Guide*. Microsoft Learn.

Microsoft. *SET STATISTICS XML (Transact-SQL)*. Microsoft Learn.

Microsoft. *Showplan Logical and Physical Operators Reference*. Microsoft Learn.

Microsoft. *SQL Server Index Design Guide*. Microsoft Learn.

Microsoft. *sys.indexes (Transact-SQL)*. Microsoft Learn.

Microsoft. *sys.index_columns (Transact-SQL)*. Microsoft Learn.

Parr, Terence. *The Definitive ANTLR 4 Reference*. Pragmatic Bookshelf, 2013.

Python Software Foundation. *Python 3 Documentation*.

pyodbc Contributors. *pyodbc Documentation*. GitHub Repository.

Sommerville, Ian. *Software Engineering*. 10th ed., Pearson, 2015.